

TPU 高速資料管道：tf.data.Dataset 和 TFRecords

來源：<https://codelabs.developers.google.com/codelabs/keras-flowers-data?hl=zh-tw>

步驟數：6

TPU 速度飛快，訓練資料的串流必須跟上訓練速度。在本研究室中，您將瞭解如何使用 tf.data.Dataset API 從 GCS 載入資料來提供 TPU。

目錄

1. [1. 總覽](#)
2. [2. Google Colaboratory 快速入門](#)
3. [3. \[INFO\] 什麼是 Tensor Processing Unit \(TPU\) ?](#)
4. [4. 正在載入資料](#)
5. [5. 快速載入資料](#)
6. [6. 恭喜 !](#)

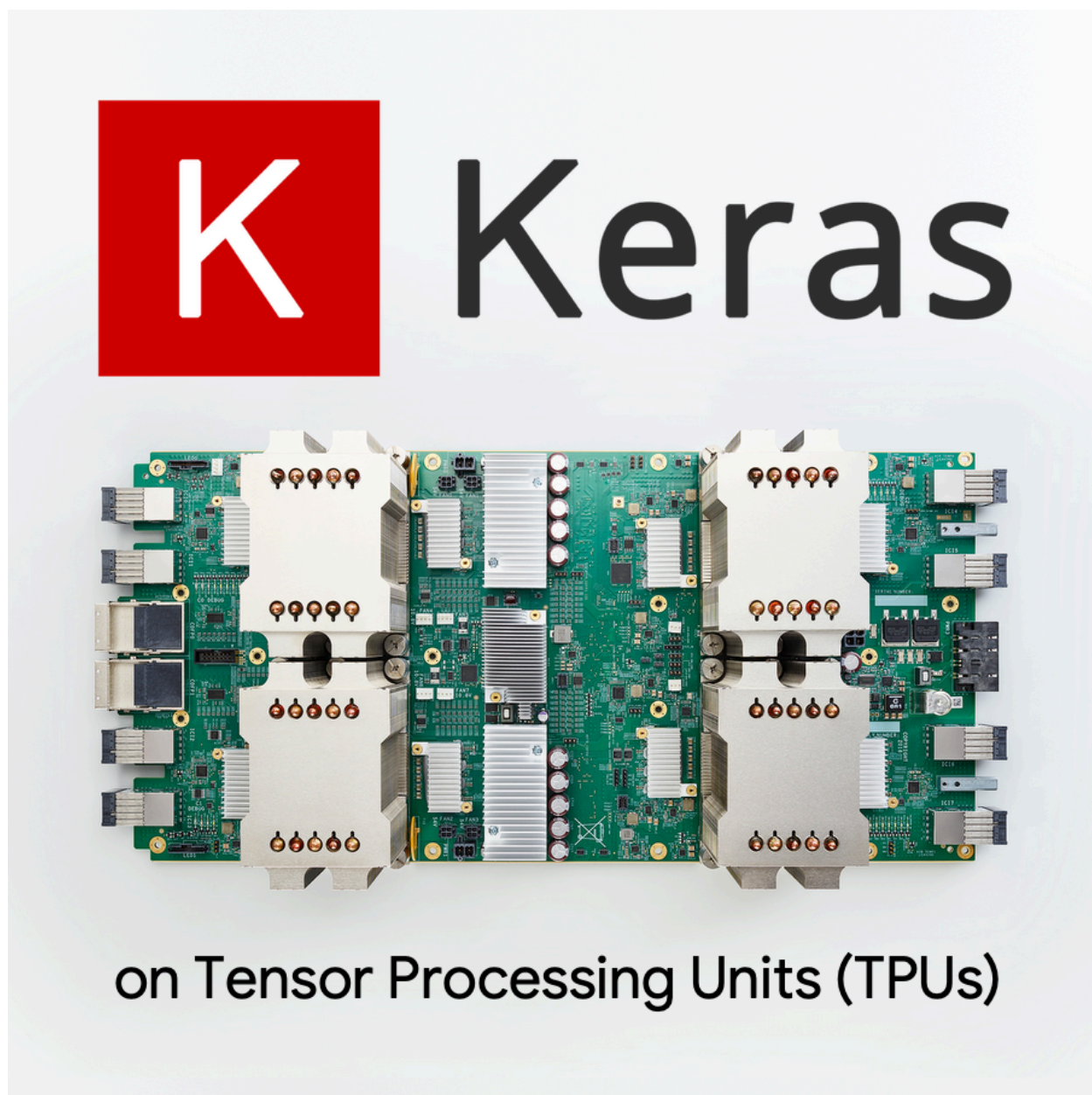
步驟 1 · 約 1 分鐘

1. 總覽

TPU 的速度非常快，訓練資料串流必須跟上訓練速度。在本實驗室中，您將瞭解如何使用 `tf.data.Dataset` API 從 GCS 載入資料，以提供 TPU。

本實驗室是「在 TPU 上使用 Keras」系列的第一部分。你可以依下列順序執行這些步驟，也可以單獨執行。

- [本實驗室] TPU 高速資料管道：`tf.data.Dataset` 和 `TFRecords`
- 建立第一個 Keras 模型 (採用遷移學習)
- 卷積類神經網路 (搭配使用 Keras 和 TPU)
- 搭配使用 Keras 和 TPU，翻新 `convnets`、`squeezenet` 和 `Xception` 模型



課程內容

- 使用 `tf.data.Dataset` API 載入訓練資料
- 使用 `TFRecord` 格式從 GCS 有效率地載入訓練資料

意見回饋

如果您發現這個程式碼研究室有任何錯誤，請告訴我們。你可以透過 GitHub 問題 [\[意見回饋連結\]](#) 提供意見。

步驟 2 · 約 2 分鐘

2. Google Colaboratory 快速入門

本實驗室使用 Google Colaboratory，您無須進行任何設定。Colaboratory 是線上筆記本平台，適用於教育用途。提供免費的 CPU、GPU 和 TPU 訓練。

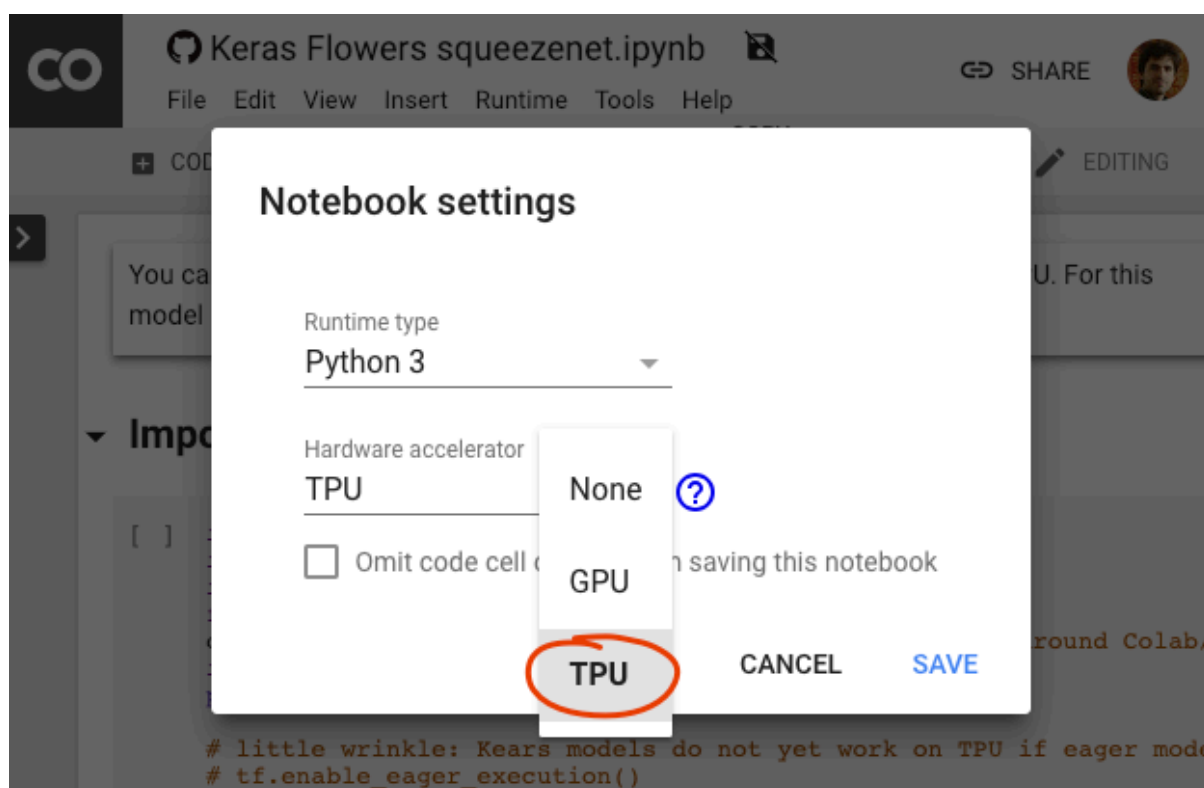
If this is old news to you **SKIP** ✕ to the next exercise otherwise **READ ON** ↘

您可以開啟這個範例筆記本，並執行幾個儲存格，熟悉 Colaboratory 的操作方式。



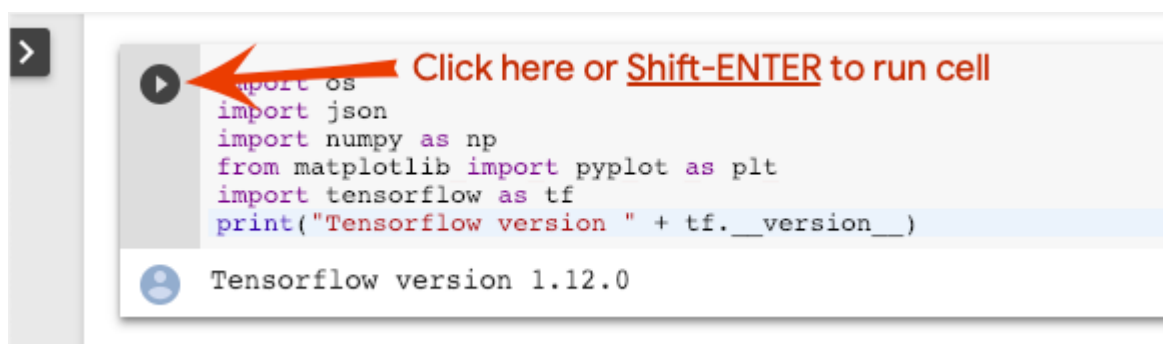
Welcome to Colab.ipynb

選取 TPU 後端



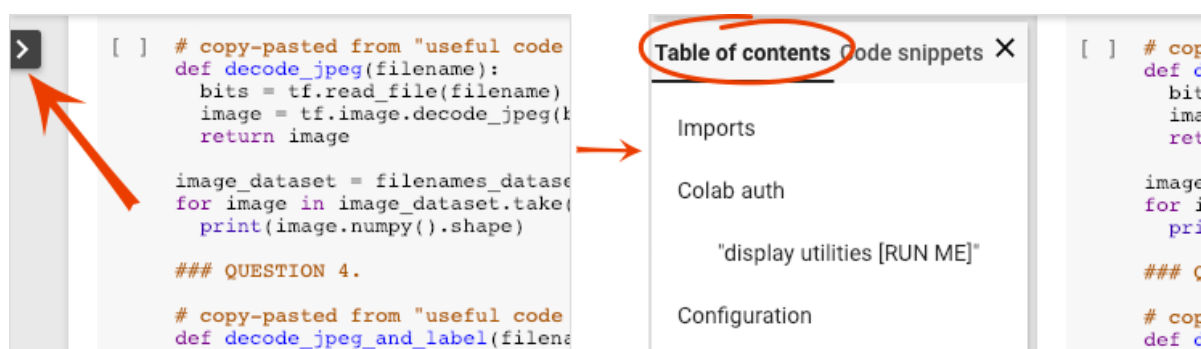
在 Colab 選單中，依序選取「執行階段」>「變更執行階段類型」，然後選取「TPU」。在本程式碼研究室中，您將使用強大的 TPU (Tensor Processing Unit)，支援硬體加速訓練。首次執行時，系統會自動連線至執行階段，您也可以使用右上角的「連線」按鈕。

筆記本執行



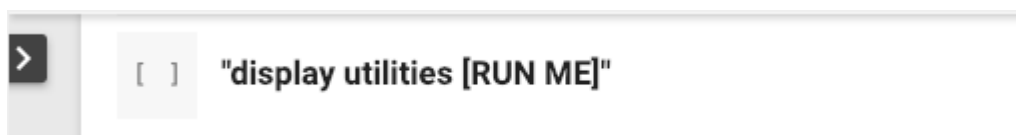
按一下儲存格並使用 Shift-ENTER，即可一次執行一個儲存格。您也可以依序選取「執行階段」>「全部執行」，執行整個筆記本。

Table of contents



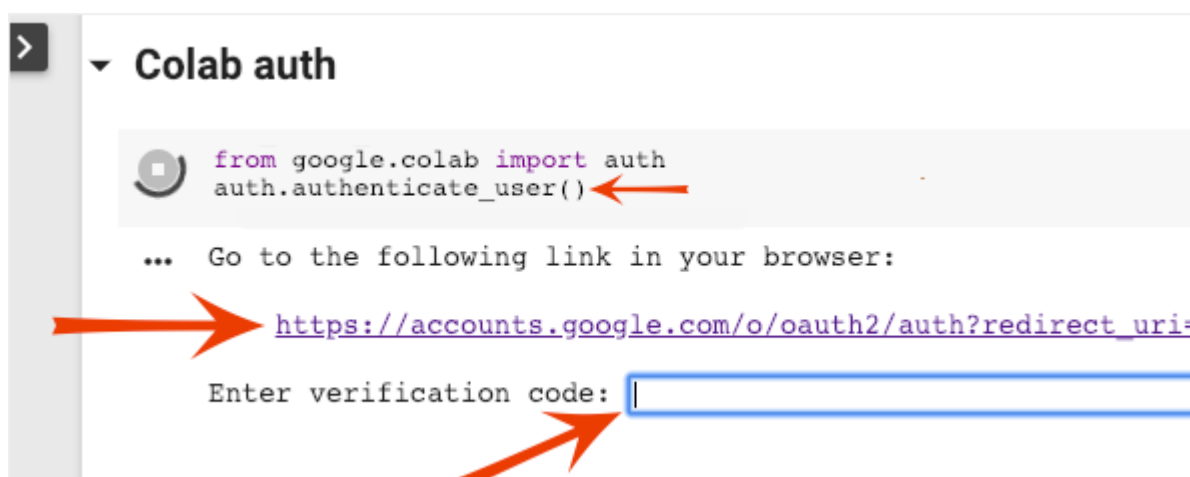
所有筆記本都有目錄。你可以使用左側的黑色箭頭開啟這個選單。

隱藏的儲存格



部分儲存格只會顯示標題，這是 Colab 專屬的筆記本功能。您可以按兩下查看其中的程式碼，但通常不會很有趣。通常是支援或視覺化函式。您仍須執行這些儲存格，才能定義其中的函式。

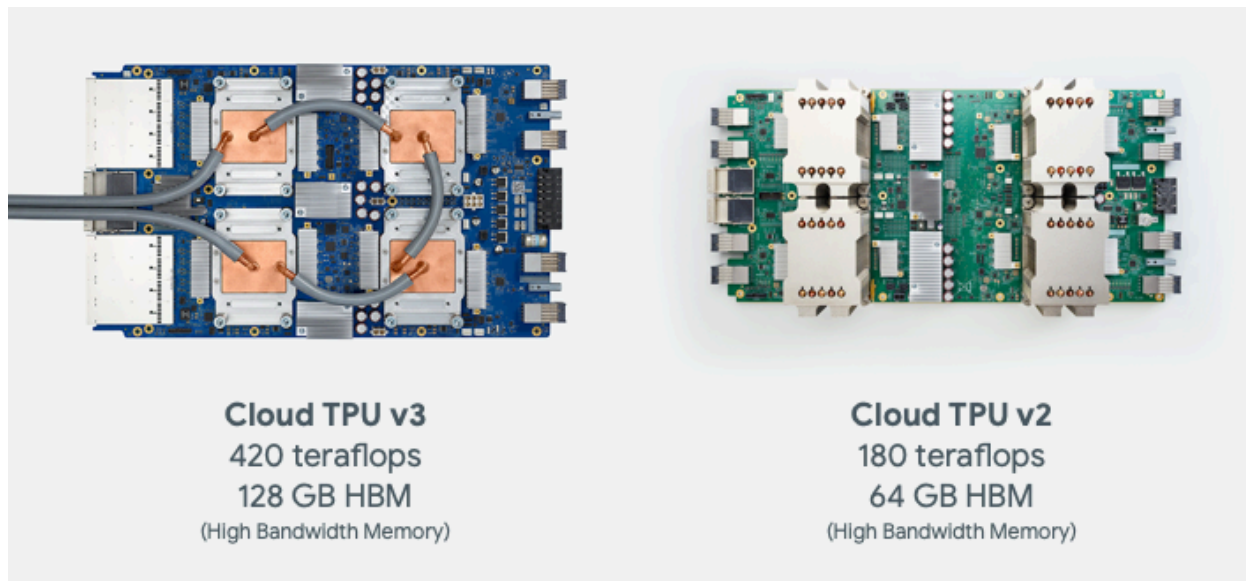
驗證



只要使用已授權的帳戶進行驗證，Colab 就能存取您的私人 Google Cloud Storage bucket。上述程式碼片段會觸發驗證程序。

3. [INFO] 什麼是 Tensor Processing Unit (TPU) ?

簡介



TPU 是專門用於深度學習任務的硬體加速器。在本程式碼研究室中，您將瞭解如何搭配 Keras 和 TensorFlow 2 使用這些工具。Cloud TPU 的基本設定為 8 個核心，此外還有多種大型設定，稱為「TPU Pod」，最多可達 2048 個核心。增加訓練批量，藉此加速訓練。

在 Keras 中於 TPU 上訓練模型的程式碼 (如果沒有 TPU，則改用 GPU 或 CPU)：

```
try: # detect TPUs
    tpu = tf.distribute.cluster_resolver.TPUClusterResolver.connect()
    strategy = tf.distribute.TPUStrategy(tpu)
except ValueError: # detect GPUs
    strategy = tf.distribute.MirroredStrategy() # for CPU/GPU or multi-GPU machines

# use TPUStrategy scope to define model
with strategy.scope():
    model = tf.keras.Sequential( ... )
    model.compile( ... )

# train model normally on a tf.data.Dataset
model.fit(training_dataset, epochs=EPOCHS, steps_per_epoch=...)
```

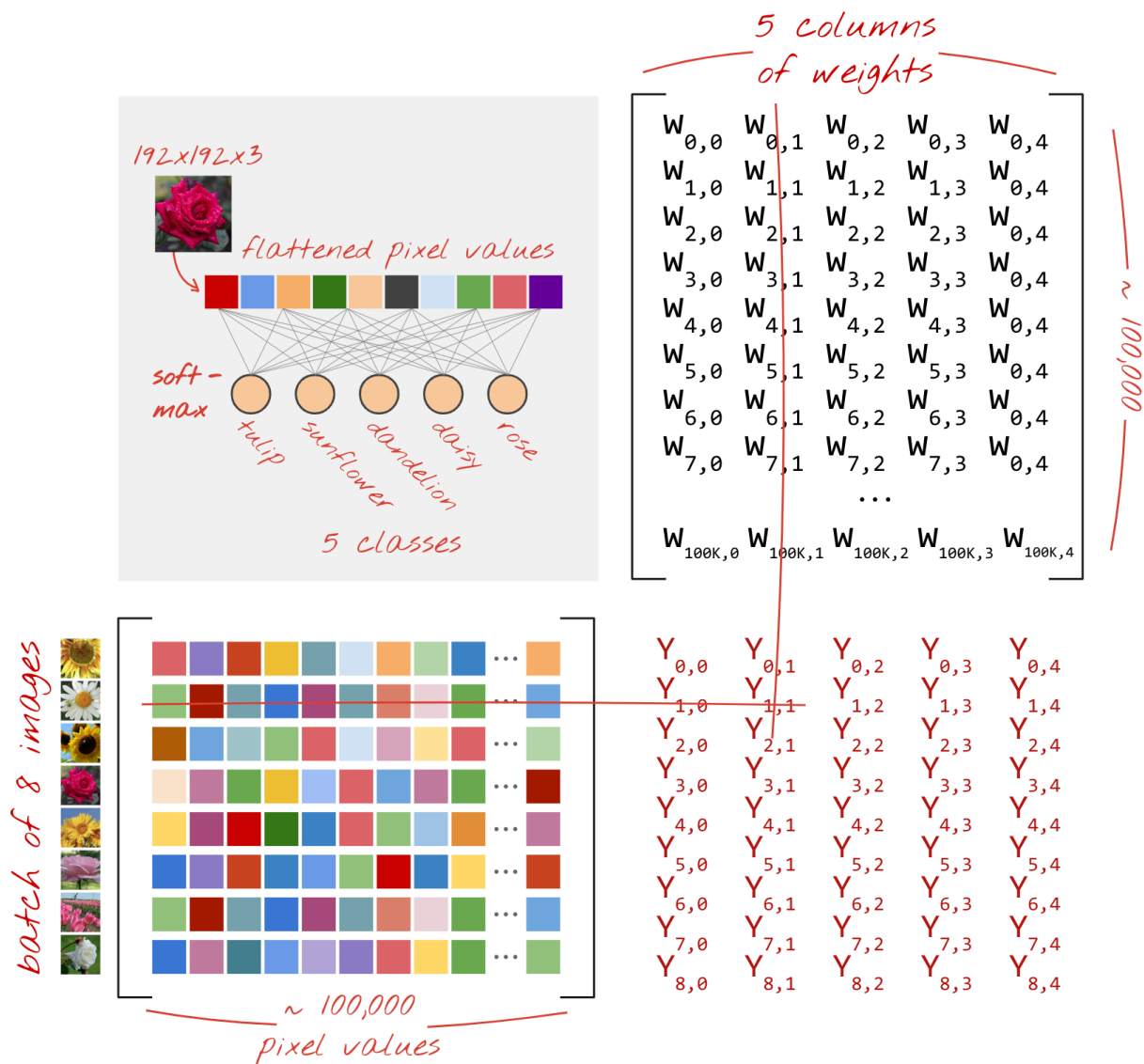
我們今天將使用 TPU，以互動式速度 (每次訓練執行幾分鐘) 建構及最佳化花朵分類器。

If this is old news to you **SKIP**  to the next exercise otherwise **READ ON**



為何選擇 TPU ?

現代 GPU 是以可程式化的「核心」為基礎架構，這種架構非常靈活，可處理各種工作，例如 3D 算繪、深度學習、物理模擬等。另一方面，TPU 會將傳統向量處理器與專用矩陣乘法單元配對，並擅長處理大型矩陣乘法運算占主導地位的任何工作，例如類神經網路。

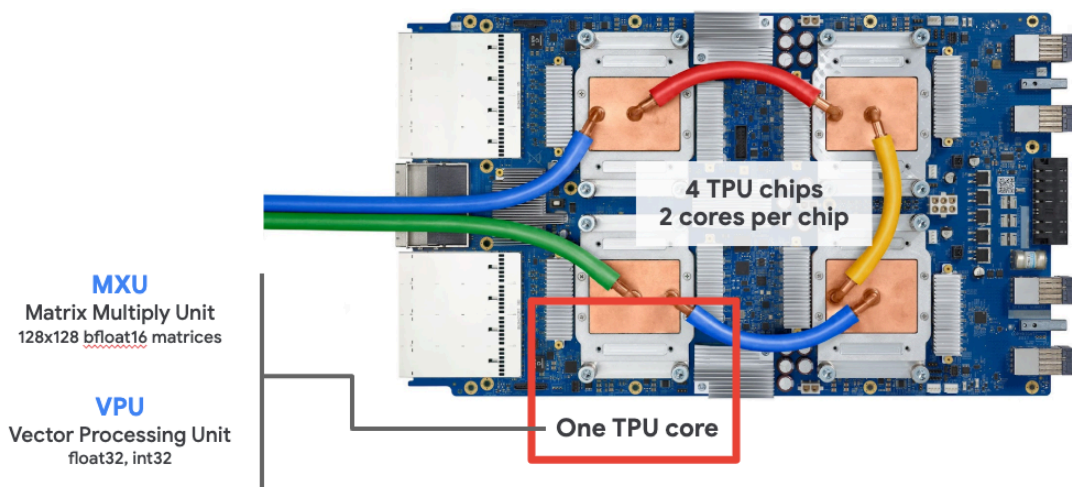


插圖：密集類神經網路層，以矩陣乘法表示，一次透過類神經網路處理一批八張圖片。請執行一行 x 欄的乘法，確認這確實是圖像所有像素值的加權總和。雖然有點複雜，但卷積層也可以表示為矩陣乘法 (請參閱第 1 節的說明)。

硬體

MXU 和 VPU

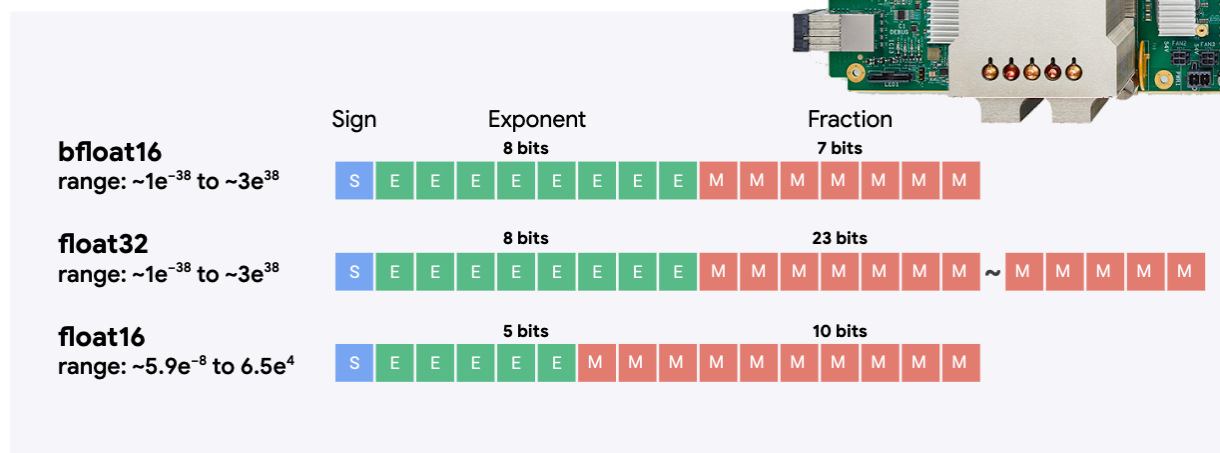
TPU v2 核心由矩陣乘法單元 (MXU) 和向量處理單元 (VPU) 組成，前者用於執行矩陣乘法，後者則用於執行所有其他工作，例如活化、softmax 等。VPU 會處理 float32 和 int32 運算。另一方面，MXU 則是以混合精度 16-32 位元浮點格式運作。



混合精確度浮點和 bfloat16

MXU 會使用 bfloat16 輸入和 float32 輸出計算矩陣乘法。中繼累計作業會以 float32 精確度執行。

bfloat16



類神經網路訓練通常能抵禦因浮點數精確度降低而產生的雜訊。在某些情況下，雜訊甚至有助於最佳化工具收斂。傳統上，16 位元浮點數精度用於加速運算，但 float16 和 float32 格式的範圍差異很大。將精確度從 float32 降為 float16 通常會導致溢位和下溢。雖然有解決方案，但通常需要額外作業才能讓 float16 正常運作。

因此，Google 在 TPU 中導入了 bfloat16 格式。bfloat16 是經過截斷的 float32，與 float32 的指數位元和範圍完全相同。此外，TPU 會以混合精度計算矩陣乘法，並使用 bfloat16 輸入，但輸出為 float32，因此通常不需要變更程式碼，就能享有精度降低帶來的效能提升。

在 TPU 上，預設會使用 bfloat16/float32 混合精確度。您不必變更 Tensorflow 程式碼，即可啟用這項功能。您仍可在整個程式碼中使用 float32。執行矩陣乘法時，TPU 會自動將輸入內容截斷為 bfloat16。產生的矩陣為 float32。

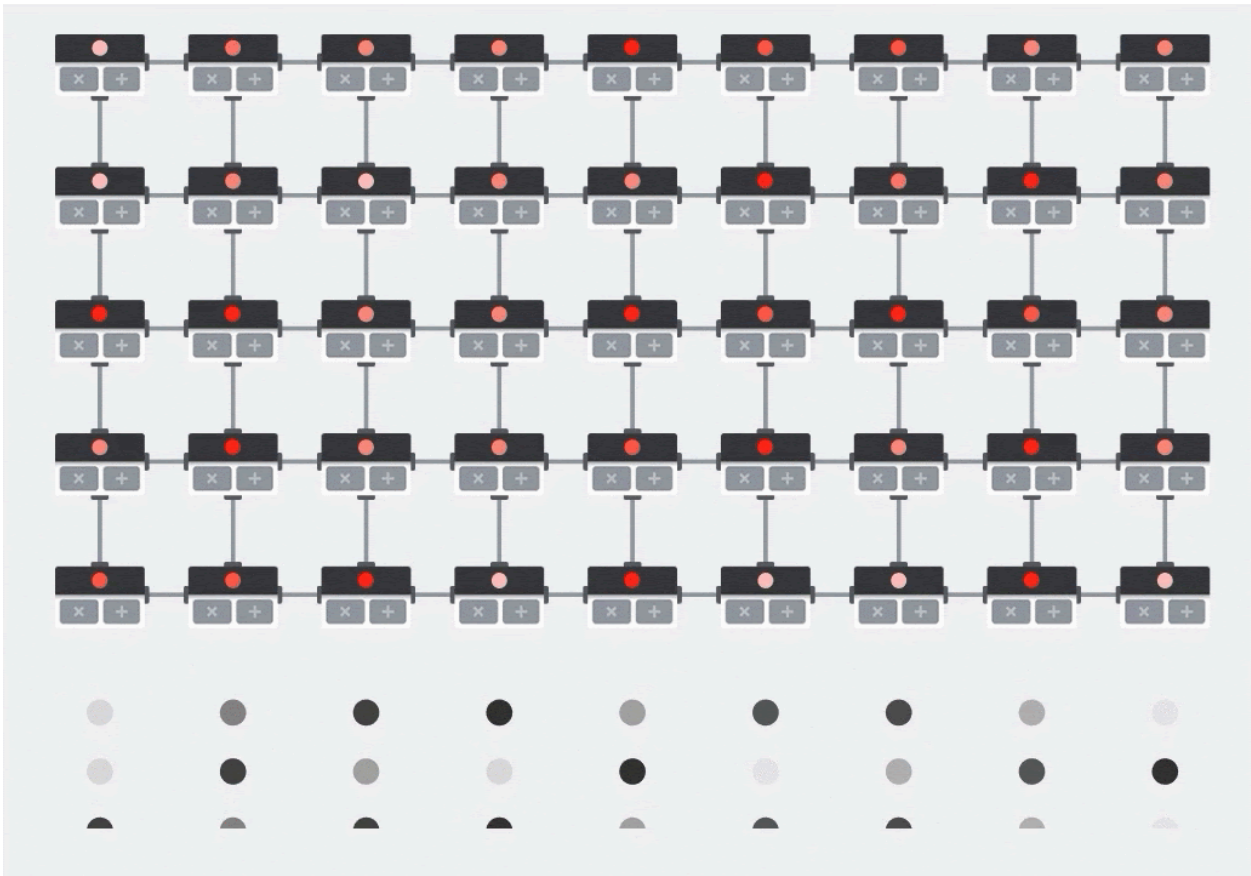
Systolic array

MXU 會使用所謂的「脈動陣列」架構，在硬體中實作矩陣乘法，讓資料元素流經硬體運算單元陣列。(在醫學上，「收縮」是指心臟收縮和血流，這裡則是指資料流動。)

矩陣乘法的基本元素是兩個矩陣中，一個矩陣的列與另一個矩陣的欄之間的點積 (請參閱本節頂端的插圖)。以矩陣乘法 $Y=X*W$ 來說，結果的一個元素會是：

$$Y[2,0] = X[2,0]*W[0,0] + X[2,1]*W[1,0] + X[2,2]*W[2,0] + \dots + X[2,n]*W[n,0]$$

在 GPU 上，您可以將這個點積程式設計到 GPU「核心」，然後在盡可能多的「核心」上平行執行，嘗試一次計算結果矩陣的每個值。如果產生的矩陣為 128x128 大小，則需要 128x128=16K 個「核心」，這通常是不可能的。最大的 GPU 約有 4000 個核心。另一方面，TPU 會為 MXU 中的運算單元使用最少的硬體：只有 `bfloat16 x bfloat16 => float32` 乘法累加器，沒有其他元件。這些單元非常小，TPU 可以在 128x128 MXU 中實作 16K 個單元，並一次處理這個矩陣乘法。

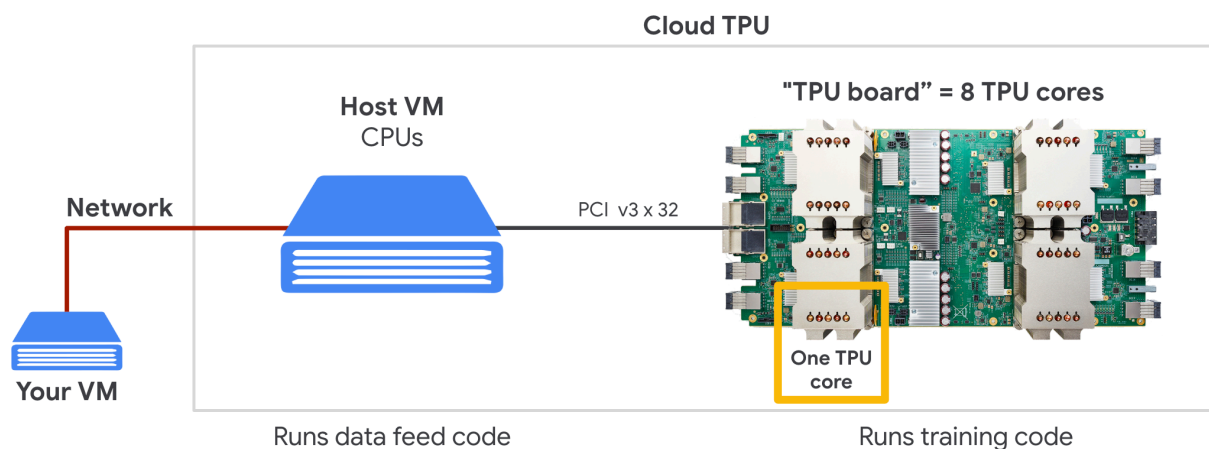


插圖：MXU 脈動陣列。運算元素是乘法累加器。一個矩陣的值會載入陣列 (紅點)。其他矩陣的值會流經陣列 (灰色點)。垂直線會將值向上傳播。水平線會傳播部分總和。使用者可自行驗證，當資料流經陣列時，右側會輸出矩陣乘法運算的結果。

此外，在 MXU 中計算點積時，中間總和只會在相鄰的運算單元之間流動。這類值不需要儲存及從記憶體或暫存器檔案中擷取。因此，在計算矩陣乘法時，TPU 脈動陣列架構在密度和電力方面具有顯著優勢，且速度也比 GPU 快上不少。

Cloud TPU

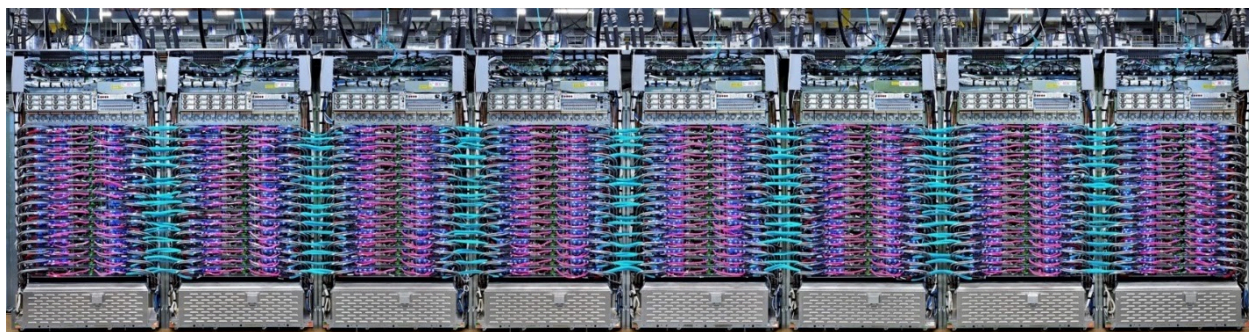
在 Google Cloud Platform 上要求「[Cloud TPU v2](#)」時，您會取得具有 PCI 連接 TPU 板的虛擬機器 (VM)。TPU 板有四個雙核心 TPU 晶片。每個 TPU 核心都具備 VPU (向量處理單元) 和 128x128 MXU (矩陣乘法單元)。這個「Cloud TPU」通常會透過網路連線至要求該 TPU 的 VM。因此，完整圖片如下所示：



插圖：您的 VM，以及透過網路連結的「Cloud TPU」加速器。「Cloud TPU」本身是由 VM 和 PCI 連接的 TPU 板組成，板上則有四個雙核心 TPU 晶片。

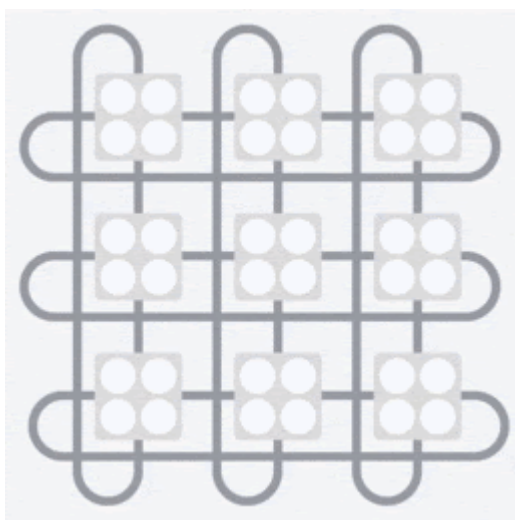
TPU Pod

在 Google 的資料中心，TPU 會連線至高效能運算 (HPC) 互連，因此可視為一個非常大的加速器。Google 將這些節點稱為 Pod，最多可包含 512 個 TPU v2 核心或 2048 個 TPU v3 核心。



插圖：TPU v3 Pod。透過 HPC 互連網路連線的 TPU 板和機架。

在訓練期間，系統會使用 all-reduce 演算法在 TPU 核心之間交換梯度 ([這裡有 all-reduce 的詳細說明](#))。訓練中的模型可透過訓練大量批次，充分運用硬體資源。



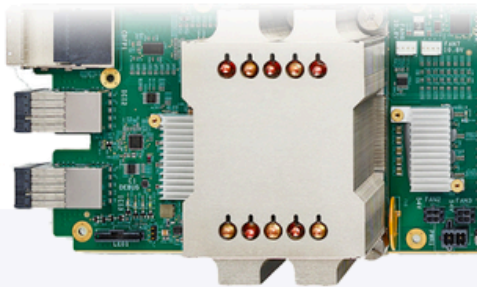
插圖：在 Google TPU 的 2D 環面網格 HPC 網路中，使用 all-reduce 演算法訓練期間的梯度同步。

軟體

大型批量訓練

TPU 的理想批量大小為每個 TPU 核心 128 個資料項目，但每個 TPU 核心 8 個資料項目時，硬體就能展現良好的使用率。請注意，一個 Cloud TPU 有 8 個核心。

在本程式碼研究室中，我們將使用 Keras API。在 Keras 中，您指定的批次是整個 TPU 的全域批量大小。系統會自動將批次分割為 8 個，並在 TPU 的 8 個核心上執行。



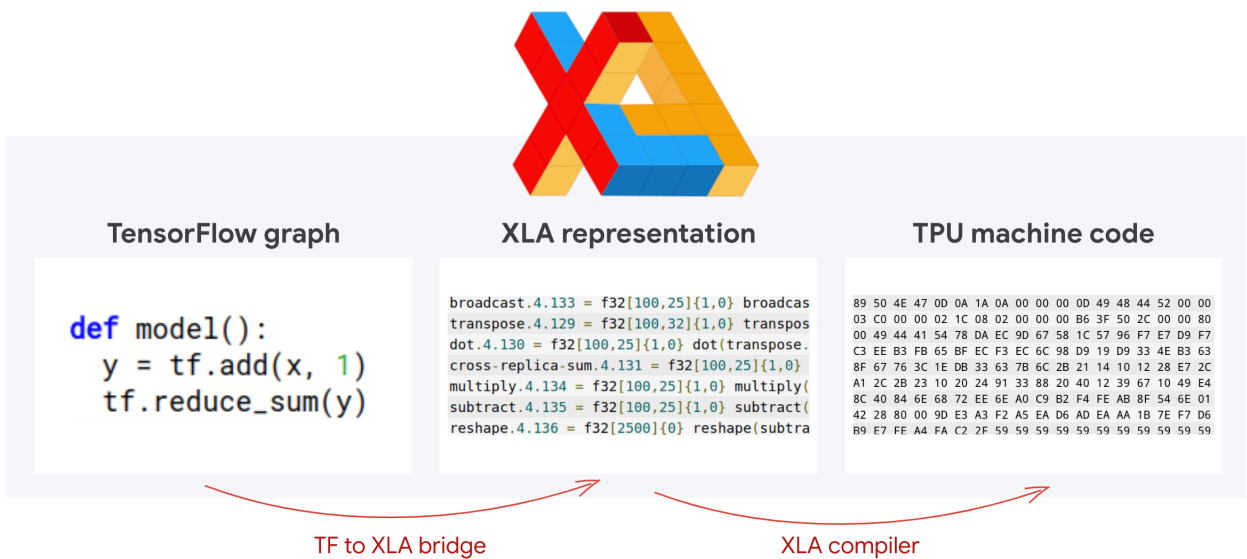
TPU batch size
ideal 128 per TPU core
interesting from 8 per core

DONE!
scales from
8 to 2048 cores

如需其他效能提示，請參閱 [TPU 效能指南](#)。如果批次大小非常大，部分模型可能需要特別注意，詳情請參閱 [LARSOptimizer](#)。

幕後花絮：XLA

Tensorflow 程式會定義運算圖形。TPU 不會直接執行 Python 程式碼，而是執行 TensorFlow 程式定義的運算圖。在幕後，名為 XLA (加速線性代數編譯器) 的編譯器會將運算節點的 Tensorflow 圖形轉換為 TPU 機器碼。這個編譯器也會對程式碼和記憶體配置執行許多進階最佳化作業。當工作傳送至 TPU 時，系統會自動進行編譯。您不必在建構鏈中明確加入 XLA。



圖解：如要在 TPU 上執行，Tensorflow 程式定義的運算圖會先轉換為 XLA (加速線性代數編譯器) 表示法，然後由 XLA 編譯為 TPU 機器碼。

在 Keras 中使用 TPU

Tensorflow 2.1 以上版本支援透過 Keras API 使用 TPU。Keras 支援 TPU 和 TPU Pod。以下範例適用於 TPU、GPU 和 CPU：

```
try: # detect TPUs
    tpu = tf.distribute.cluster_resolver.TPUClusterResolver.connect()
    strategy = tf.distribute.TPUStrategy(tpu)
except ValueError: # detect GPUs
    strategy = tf.distribute.MirroredStrategy() # for CPU/GPU or multi-GPU machines

# use TPUStrategy scope to define model
with strategy.scope():
    model = tf.keras.Sequential( ... )
    model.compile( ... )

# train model normally on a tf.data.Dataset
model.fit(training_dataset, epochs=EPOCHS, steps_per_epoch=...)
```

在這個程式碼片段中：

- `TPUClusterResolver().connect()` 會在網路上尋找 TPU。在大多數 Google Cloud 系統 (AI Platform 工作、Colaboratory、Kubeflow、透過「ctpu up」公用程式建立的深度學習 VM) 上，這項工具都能在沒有參數的情況下運作。這些系統會透過 `TPU_NAME` 環境變數得知 TPU 所在位置。如要手動建立 TPU，請在您使用的 VM 上設定 `TPU_NAME` 環境變數，或使用明確的參數呼叫 `TPUClusterResolver`：`TPUClusterResolver(tp_uname, zone, project)`
- `TPUStrategy` 是實作分配和「全縮減」梯度同步演算法的部分。
- 這項策略是透過範圍套用。模型必須在策略範圍() 內定義。
- `tpu_model.fit` 函式預期會收到 `tf.data.Dataset` 物件，做為 TPU 訓練的輸入內容。

常見的 TPU 移植作業

- 雖然在 TensorFlow 模型中載入資料的方法有很多種，但使用 TPU 時，必須使用 `tf.data.Dataset` API。
- TPU 的速度非常快，因此在 TPU 上執行時，擷取資料通常會成為瓶頸。您可以使用 [TPU 效能指南](#) 中的工具，偵測資料瓶頸和其他效能提示。
- 系統會將 `int8` 或 `int16` 數字視為 `int32`。TPU 沒有以少於 32 位元運作的整數硬體。
- 系統不支援部分 TensorFlow 作業。[請參閱這份清單](#)。好消息是，這項限制只適用於訓練程式碼，也就是透過模型的前向和後向傳遞。您仍可在資料輸入管道中使用所有 TensorFlow 作業，因為這些作業會在 CPU 上執行。
- TPU 不支援 `tf.py_func`。

步驟 4 · 約 10 分鐘

4. 正在載入資料





799 Tulips
699 Sunflowers
641 Roses
633 Daisies
898 Dandelions

3670 flowers

我們將使用花卉圖片資料集。目標是學會將這些花朵分類為 5 種花卉類型。資料載入作業是使用 `tf.data.Dataset` API 執行。首先，請先瞭解 API。

實作實驗室

請開啟下列筆記本、執行儲存格 (Shift-ENTER)，並按照標示「WORK REQUIRED」的指示操作。



[Fun with tf.data.Dataset \(playground\).ipynb](#)

其他資訊

關於「flowers」資料集

資料集分為 5 個資料夾。每個資料夾都包含一種花朵。資料夾名稱分別為向日葵、雛菊、蒲公英、鬱金香和玫瑰。資料託管在 Google Cloud Storage 的公開 bucket 中。摘錄：

```
gs://flowers-public/sunflowers/5139971615_434ff8ed8b_n.jpg
gs://flowers-public/daisy/8094774544_35465c1c64.jpg
gs://flowers-public/sunflowers/9309473873_9d62b9082e.jpg
gs://flowers-public/dandelion/19551343954_83bb52f310_m.jpg
gs://flowers-public/dandelion/14199664556_188b37e51e.jpg
gs://flowers-public/tulips/4290566894_c7f061583d_m.jpg
gs://flowers-public/roses/3065719996_c16ecd5551.jpg
gs://flowers-public/dandelion/8168031302_6e36f39d87.jpg
gs://flowers-public/sunflowers/9564240106_0577e919da_n.jpg
gs://flowers-public/daisy/14167543177_cd36b54ac6_n.jpg
```


為什麼要使用 `tf.data.Dataset` ？

Keras 和 TensorFlow 的所有訓練和評估函式都會接受資料集。在資料集中載入資料後，API 會提供所有適用於類神經網路訓練資料的常見功能：

```
dataset = ... # load something (see below)
dataset = dataset.shuffle(1000) # shuffle the dataset with a buffer of 1000
dataset = dataset.cache() # cache the dataset in RAM or on disk
dataset = dataset.repeat() # repeat the dataset indefinitely
dataset = dataset.batch(128) # batch data elements together in batches of 128
AUTOTUNE = tf.data.AUTOTUNE
dataset = dataset.prefetch(AUTOTUNE) # prefetch next batch(es) while training
```

如需成效提示和資料集最佳做法，請參閱[這篇文章](#)。請參閱[這份參考文件](#)。

`tf.data.Dataset` 基礎知識

資料通常會以多個檔案的形式提供，這裡是指圖片。您可以呼叫下列項目，建立檔案名稱資料集：

```
filenames_dataset = tf.data.Dataset.list_files('gs://flowers-public/**/*.jpg')
# The parameter is a "glob" pattern that supports the * and ? wildcards.
```

接著，您會將函式「對應」至每個檔案名稱，這通常會將檔案載入並解碼為記憶體中的實際資料：

```
def decode_jpeg(filename):
    bits = tf.io.read_file(filename)
    image = tf.io.decode_jpeg(bits)
    return image

image_dataset = filenames_dataset.map(decode_jpeg)
# this is now a dataset of decoded images (uint8 RGB format)
```

如要疊代資料集，請按照下列步驟操作：

```
for data in my_dataset:
    print(data)
```

元組資料集

在監督式學習中，訓練資料集通常由訓練資料和正確答案配對組成。如要允許這項操作，解碼函式可以傳回元組。接著，您會取得元組資料集，並在疊代時傳回元組。傳回的值是 Tensorflow 張量，可供模型使用。您可以對這些值呼叫 `.numpy()`，查看原始值：

```
def decode_jpeg_and_label(filename):
    bits = tf.read_file(filename)
    image = tf.io.decode_jpeg(bits)
    label = ... # extract flower name from folder name
    return image, label

image_dataset = filenames_dataset.map(decode_jpeg_and_label)
# this is now a dataset of (image, label) pairs

for image, label in dataset:
    print(image.numpy().shape, label.numpy())
```

結論：逐一載入圖片的速度很慢！

反覆處理這個資料集時，您會發現每秒可載入 1 到 2 張圖片。這太慢了！我們用於訓練的硬體加速器可維持這個速率的許多倍。請參閱下一節，瞭解如何達成這個目標。

解決方案

解決方案筆記本在此。如果遇到困難，可以參考這項工具。



[Fun with tf.data.Dataset \(solution\).ipynb](#)

涵蓋內容

- 🤔 `tf.data.Dataset.list_files`
- 🤔 `tf.data.Dataset.map`
- 🤔 元組資料集
- 😊 逐一查看資料集

請花點時間在腦中瀏覽這份檢查清單。

5. 快速載入資料

我們將在本實驗室中使用的 Tensor Processing Unit (TPU) 硬體加速器速度非常快。挑戰通常在於如何快速提供資料，讓這些加速器保持忙碌。Google Cloud Storage (GCS) 能夠維持非常高的處理量，但與所有雲端儲存系統一樣，啟動連線會產生一些網路來回傳輸費用。因此，將資料儲存為數千個個別檔案並非理想做法。我們將這些資料分批存入較少數量的檔案，並運用 `tf.data.Dataset` 的強大功能，平行讀取多個檔案。

朗讀

下列筆記本中的程式碼會載入圖片檔案、將圖片調整為相同大小，然後儲存在 16 個 TFRecord 檔案中。請快速瀏覽。由於本程式碼研究室的其餘部分會提供格式正確的 TFRecord 資料，因此不需要執行這項作業。



[Flower pictures to TFRecords.ipynb](#)

理想的資料版面配置，可達到最佳 GCS 處理量

經驗法則是將資料分割成數個 (數十到數百個) 較大的檔案 (數十到數百 MB)。如果檔案數量過多 (例如數千個)，存取每個檔案的時間可能會開始受到影響。如果檔案太少 (例如只有一或兩個)，就無法享有平行串流多個檔案的好處。

TFRecord 檔案格式

Tensorflow 偏好的資料儲存檔案格式是以 [protobuf](#) 為基礎的 TFRecord 格式。其他序列化格式也適用，但您可以編寫下列程式碼，直接從 TFRecord 檔案載入資料集：

```
filenames = tf.io.gfile.glob(FILENAME_PATTERN)
dataset = tf.data.TFRecordDataset(filenames)
dataset = dataset.map(...) # do the TFRecord decoding here - see below
```

為達到最佳效能，建議使用下列較複雜的程式碼，一次讀取多個 TFRecord 檔案。這段程式碼會平行讀取 N 個檔案，並忽略資料順序，以提高讀取速度。

```
AUTOTUNE = tf.data.AUTOTUNE
ignore_order = tf.data.Options()
ignore_order.experimental_deterministic = False

filenames = tf.io.gfile.glob(FILENAME_PATTERN)
dataset = tf.data.TFRecordDataset(filenames, num_parallel_reads=AUTOTUNE)
dataset = dataset.with_options(ignore_order)
dataset = dataset.map(...) # do the TFRecord decoding here - see below
```

TFRecord 一覽表

TFRecord 可儲存三種資料：**位元組字串** (位元組清單)、**64 位元整數**和 **32 位元浮點數**。這些值一律會儲存為清單，單一資料元素則會是大小為 1 的清單。您可以使用下列輔助函式，將資料儲存到 TFRecord。

寫入位元組字串

```
# warning, the input is a list of byte strings, which are themselves lists of bytes
def _bytestring_feature(list_of_bytestrings):
    return tf.train.Feature(bytes_list=tf.train.BytesList(value=list_of_bytestrings))
```

寫入整數

```
def _int_feature(list_of_ints): # int64
    return tf.train.Feature(int64_list=tf.train.Int64List(value=list_of_ints))
```

撰寫浮動文字

```
def _float_feature(list_of_floats): # float32
    return tf.train.Feature(float_list=tf.train.FloatList(value=list_of_floats))
```

編寫 TFRecord，使用上述輔助程式

```
# input data in my_img_bytes, my_class, my_height, my_width, my_floats
with tf.python_io.TFRecordWriter(filename) as out_file:
    feature = {
        "image": _bytestring_feature([my_img_bytes]), # one image in the list
        "class": _int_feature([my_class]),             # one class in the list
        "size": _int_feature([my_height, my_width]),   # fixed length (2) list of ints
        "float_data": _float_feature(my_floats)        # variable length list of floats
    }
    tf_record = tf.train.Example(features=tf.train.Features(feature=feature))
    out_file.write(tf_record.SerializeToString())
```

如要從 TFRecord 讀取資料，您必須先宣告所儲存記錄的版面配置。在宣告中，您可以將任何具名欄位當做固定長度清單或可變長度清單存取：

從 TFRecords 讀取資料

```
def read_tfrecord(data):
    features = {
        # tf.string = byte string (not text string)
        "image": tf.io.FixedLenFeature([], tf.string), # shape [] means scalar, here, a single
byte string
        "class": tf.io.FixedLenFeature([], tf.int64), # shape [] means scalar, i.e. a single
item
        "size": tf.io.FixedLenFeature([2], tf.int64), # two integers
        "float_data": tf.io.VarLenFeature(tf.float32) # a variable number of floats
    }

    # decode the TFRecord
    tf_record = tf.io.parse_single_example(data, features)

    # FixedLenFeature fields are now ready to use
    sz = tf_record['size']

    # Typical code for decoding compressed images
    image = tf.io.decode_jpeg(tf_record['image'], channels=3)

    # VarLenFeature fields require additional sparse.to_dense decoding
    float_data = tf.sparse.to_dense(tf_record['float_data'])

    return image, sz, float_data

# decoding a tf.data.TFRecordDataset
dataset = dataset.map(read_tfrecord)
# now a dataset of triplets (image, sz, float_data)
```

實用的程式碼片段：

讀取單一資料元素

```
tf.io.FixedLenFeature([], tf.string)    # for one byte string
tf.io.FixedLenFeature([], tf.int64)     # for one int
tf.io.FixedLenFeature([], tf.float32)   # for one float
```

讀取元素固定大小的清單

```
tf.io.FixedLenFeature([N], tf.string)   # list of N byte strings
tf.io.FixedLenFeature([N], tf.int64)    # list of N ints
tf.io.FixedLenFeature([N], tf.float32)  # list of N floats
```

讀取數量不定的資料項目

```
tf.io.VarLenFeature(tf.string)          # list of byte strings
tf.io.VarLenFeature(tf.int64)           # list of ints
tf.io.VarLenFeature(tf.float32)         # list of floats
```

VarLenFeature 會傳回稀疏向量，因此解碼 TFRecord 後還需要額外步驟：

```
dense_data = tf.sparse.to_dense(tf_record['my_var_len_feature'])
```

TFRecord 中也可以有選填欄位。如果在讀取欄位時指定預設值，當欄位遺失時，系統會傳回預設值，而非錯誤。

```
tf.io.FixedLenFeature([], tf.int64, default_value=0) # this field is optional
```

涵蓋內容

- 🤖 分片處理資料檔案，以便從 GCS 快速存取
- 🧐 如何編寫 TFRecords。(您已經忘記語法了嗎？沒關係，請將這個頁面加入書籤，當做快速參考資料)
- 😊 使用 TFRecordDataset 從 TFRecords 載入資料集

請花點時間在腦中瀏覽這份檢查清單。

步驟 6 · 約 0 分鐘

6. 恭喜！

現在你可以將資料提供給 TPU。請繼續下一個實驗室

- [本實驗室] **TPU 高速資料管道**：`tf.data.Dataset` 和 `TFRecords`
- [建立第一個 Keras 模型 \(採用遷移學習\)](#)
- [卷積類神經網路 \(搭配使用 Keras 和 TPU\)](#)
- [搭配使用 Keras 和 TPU，翻新 convnets、squeezenet 和 Xception 模型](#)

TPU 實務

TPU 和 GPU 可在 [Cloud AI Platform](#) 上使用：

- 在 [Deep Learning VM](#) 上
- 在 [AI 平台筆記本](#) 中
- 在 [AI Platform Training](#) 工作中

最後，我們非常重視意見回饋。如果您在本實驗室中發現任何錯誤，或認為有需要改進之處，請告訴我們。你可以透過 GitHub 問題 [\[意見回饋連結\]](#) 提供意見。



作者：Martin
Görner
Twitter：
[@martin_gorner](#)



TensorFlow